# CONFORMANCE



# ARI Conformance Registry

A runtime is **ARI-conformant** when it passes the conformance kit for a named profile, at a named spec + kit version. This file is the public record of claims. Conformance is *earned by evidence*, not granted — see ARI §10.4 and GOVERNANCE.md.

## How to claim conformance

1. Run the kit against your runtime:

   ```
   pytest ari-conformance/ -v
   ```

   (or port the kit's checks to your language — the requirements are language-neutral).

2. Record the result in the required format:

   > **ARI** `<spec-version>` · `<profile(s)>` · **kit** `<kit-version>` · `<pass>/<total>`

   Example: *ARI 0.1 · Core + Embedded · kit v0.1 · 24/24*

3. Open a PR adding a row to the table below, linking your CI run or results log.

**Do not** advertise "ARI-compatible" without a named profile and version — that claim is meaningless and is disallowed by the spec.

## Registry

| Runtime | Language | Profiles | Spec | Kit | Result | Evidenc |
|---|---|---|---|---|---|---|
| marrow | C++ / Python | Core, Embedded (structural) | 0.1 | v0.1 | pending first green CI run | conformar CI job |

> marrow's structural ARI-Embedded checks pass locally without a build; the extension-dependent Core/Embedded checks run in the `conformance` CI job. The row above will be updated with the concrete pass count once that job is green and the gate is flipped from informational to required.

## Profiles at a glance

| Profile | What it asserts | Spec |
|---|---|---|
| | message model, provider, agent state, tools, | |

| | | |
|---|---|---|
| **ARI-Core** | routing, lifecycle | §10.1 |
| **ARI-Embedded** | native-embeddable, bounded memory, strict redaction, mandatory cancel/timeout | §10.2 |
| **ARI-Server** | streaming, durable persistence, observability | §10.3 |